

XPath Functions Cheat Sheet - Playwright Edition

A comprehensive guide to XPath functions that work in Playwright, Selenium, and Appium.

1. BASIC XPATH SYNTAX

Absolute vs Relative Paths

/html/body/div/input	# Absolute path (from root)
//input	# Relative path (any element)
//div[@id='myId']/input	# Mixed path

2. XPATH AXES

Axis	Description	Example
/	Direct child	//div/input
//	Any descendant	//div//input
..	Parent element	//input/..
@	Attribute	//div[@id='test']
ancestor::	Ancestor elements	//input/ancestor::form
ancestor-or-self::	Ancestor or current	//input/ancestor-or-self::div
descendant::	All descendants	//form/descendant::input
descendant-or-self::	Descendants or current	//div/descendant-or-self::span
following::	Following elements	//input/following::button
following-sibling::	Following siblings	//input/following-sibling::button
preceding::	Preceding elements	//button/preceding::input
preceding-sibling::	Preceding siblings	//button/preceding-sibling::input
self::	Current node	//div/self::div
child::	Direct children	//form/child::input

3. XPATH PREDICATES (Conditions)

Basic Predicates

//input[1]	# First input element
//input[last()]	# Last input element
//input[last()-1]	# Second to last
//input[position()>2]	# Position greater than 2
//input[@id]	# Elements with id attribute
//input[@id='firstname']	# Exact attribute match

String Predicates

//input[@id='firstName']	# Case-sensitive match
//button[text()='Click Me']	# Exact text match
//button[text()='Login']	# Exact text
//div[contains(@id, 'test')]	# Contains text in attribute
//button[contains(text(), 'Click')]	# Contains text
//input[starts-with(@id, 'first')]	# Starts with
//input[ends-with(@id, 'Name')]	# Ends with (XPath 2.0+)
//button[normalize-space()='Click Me']	# Normalized whitespace match

Multiple Conditions

//input[@type='text' and @name='username']	# AND
//input[@type='text' or @type='email']	# OR
//button[not(@disabled)]	# NOT
//div[@class and @id]	# Has both attributes

4. XPATH STRING FUNCTIONS

text()

//button[text()='Submit']	# Exact text match
//button[text()='Sub']	# Partial (must be exact child text)

contains()

//button[contains(text(), 'Submit')]	# Text contains 'Submit'
//input[contains(@class, 'form-')]	# Class contains 'form-'
//div[contains(@id, 'myId')]	# ID contains 'myId'

starts-with()

//input[starts-with(@id, 'first')]	# ID starts with 'first'
//button[starts-with(text(), 'Click')]	# Text starts with 'Click'
//div[starts-with(@class, 'btn-')]	# Class starts with 'btn-'

ends-with()

```
//input[ends-with(@id, 'Name')]      # ID ends with 'Name'  
//button[ends-with(text(), 'mit')]  # Text ends with 'mit'
```

substring()

```
//input[substring(@id, 1, 5)='first'] # First 5 chars equal 'first'  
//button[substring(text(), 1, 4)='Click'] # First 4 chars of text  
//div[substring(@id, string-length(@id)-2)='123'] # Last 3 chars
```

substring-after()

```
//input[substring-after(@id, 'first')] # Text after 'first'  
//div[substring-after(@class, 'btn-')] # Class after 'btn-'
```

substring-before()

```
//input[substring-before(@id, 'Name')] # Text before 'Name'  
//div[substring-before(@class, '-')] # Class before first '-'
```

string-length()

```
//input[string-length(@id)=5]      # Attribute exactly 5 chars  
//button[string-length(text())>10] # Text longer than 10 chars
```

normalize-space()

```
//button[normalize-space()='Click Me'] # Removes leading/trailing spaces  
//div[normalize-space(text())='Hello'] # Normalized exact match
```

translate()

```
//input[translate(@id, 'ABC', 'abc')='firstname'] # Case conversion
```

concat()

```
//div[@id=concat('user_', '123')] # Concatenate strings
```

lower-case() / upper-case() (XPath 2.0+)

```
//button[lower-case(text())='submit'] # Case-insensitive  
//input[upper-case(@id)='FIRSTNAME'] # Upper case comparison
```

5. XPATH NUMERIC FUNCTIONS

count()

```
//form[count(input)=5]      # Form with exactly 5 inputs  
//div[count(./button)>2]    # Div with more than 2 buttons  
//*[count(*)=0]            # Elements with no children
```

sum()

```
//*[sum(@data-value)>100]    # Sum of attribute values > 100
```

floor() / ceiling() / round()

```
//*[floor(count(*)) >= 3]    # Floor of element count  
//*[ceiling(@price) > 100]   # Ceiling of attribute value  
//*[round(@rating) = 5]      # Round rating value
```

6. XPATH BOOLEAN FUNCTIONS

boolean()

```
//*[boolean(@id)]            # Elements with id attribute  
//*[not(boolean(@disabled))]
```

true() / false()

```
//input[boolean(true)]        # Always true condition  
//button[@disabled=false]     # Not disabled (if explicit)
```

7. ATTRIBUTE SELECTORS

```
/**[@id='myId'] # By ID
/**[@class='myClass'] # By class (exact match)
/**[@class='myClass other'] # By multiple classes
//input[@type='text'] # By type
//input[@placeholder='Enter name'] # By placeholder
//button[@aria-label='Submit'] # By ARIA attribute
/**[@data-test='button'] # By data attribute
/**[@name='username'] # By name
```

8. MULTIPLE ATTRIBUTE CONDITIONS

```
//input[@type='text' and @name='username'] # AND
//input[@type='text' and @placeholder='Enter text'] # Multiple conditions
//input[@type='email' or @type='text'] # OR
//button[@class and @id] # Has both attributes
//input[not(@disabled)] # NOT disabled
//input[not(@disabled) and @type='text'] # NOT AND
```

9. CLASS ATTRIBUTE PATTERNS

```
/**[@class='btn'] # Exact class match
/**[contains(@class, 'btn')] # Contains class
/**[contains(@class, 'btn-primary')] # Exact class within classes
/**[starts-with(@class, 'btn-')] # Starts with
/**[ends-with(@class, '-primary')] # Ends with
```

For Multiple Classes (Word Match)

```
/**[contains(concat(' ', @class, ' '), ' btn ')] # Word boundary
/**[contains(concat(' ', normalize-space(@class), ' '), ' active ')]
```

10. INDEX SELECTORS

```
//button[1] # First button
//button[last()] # Last button
//button[last()-1] # Second to last
//button[position()=2] # Second button
//button[position() > 3] # After position 3
//button[position() < 5] # Before position 5
(//button)[1] # First button in document (with parentheses)
(//button)[last()] # Last button in document
```

11. TEXT CONTENT SELECTORS

```
//button[text()='Click Me'] # Exact text match
//button[text()='Click Me' or text()='Submit'] # Multiple text options
/**[text() contains(., 'Partial')]] # Element contains text
//div[.='Hello World'] # Direct text content
//div[normalize-space()='Hello World'] # Normalized exact text
/**[contains(., 'Search')] # Anywhere in text content
```

Case-Insensitive Text (XPath 2.0+)

```
//button[lower-case(text()='click me')] # Case insensitive
//button[contains(lower-case(text()), 'submit')]
```

12. WILDCARD PATTERNS

```
/** # Any element
/**[@id] # Any element with id
/**[@class] # Any element with class
/**/button # Button in any parent
/**[name()='button'] # Elements with tag name 'button'
/**[name()='button' or name()='a'] # Multiple tag types
```

13. DYNAMIC ELEMENT PATTERNS

```
//input[@id='firstName'] # Static element
//input[generate-id()='xyz'] # By generated ID (rare)
/**[contains(@data-testid, 'email')] # By data attribute
//div[@id[starts-with(., 'modal')]] # Dynamic IDs
//button[@aria-label[contains(., 'Close')]] # Dynamic ARIA
```

14. FORM ELEMENT SELECTORS

//input[@type='text']	# Text inputs
//input[@type='email']	# Email inputs
//input[@type='password']	# Password inputs
//input[@type='checkbox']	# Checkboxes
//input[@type='radio']	# Radio buttons
//textarea	# Textareas
//select	# Dropdowns
//button[@type='submit']	# Submit buttons
//button[not(@type)]	# Buttons without type

15. COMPLEX SELECTOR PATTERNS

//form[@id='loginForm']//input[@type='text'][1]	# Nested with index
//div[@class='container']//button[contains(text(), 'Login')]	# Nested with text
(//button[contains(@class, 'primary')])[1]	# First matching button
//div[@id='modal']//form//input[@name='username']	# Deep nesting
//input[@required][not(@disabled)][@type='text']	# Multiple conditions

16. RELATIVE SELECTORS (Parent/Sibling)

//button[@id='submit']/..	# Parent of button
//input[@name='email']/following-sibling::button	# Next sibling button
//input[@name='email']/preceding-sibling::label	# Previous sibling
//button[@id='submit']/ancestor::form	# Ancestor form
//button[@id='submit']/ancestor::div[@class='modal']	# Specific ancestor
//button[@id='submit']/ancestor::div[1]	# First ancestor div
//input[@type='email']/parent::div	# Direct parent

17. CONTAINS WITH SPECIAL CHARACTERS

//div[contains(@id, '-')]	# Contains hyphen
//div[contains(@class, '_')]	# Contains underscore
//button[contains(text(), '@')]	# Contains special char
//div[contains(@data-value, '&')]	# Contains ampersand

18. PRACTICAL EXAMPLES FOR WEB TESTING

Login Form

//input[@name='username']	# Username field
//input[@type='password']	# Password field
//button[contains(text(), 'Login')]	# Login button
//form[@id='loginForm']//input	# All inputs in login form

Modal/Dialog

//div[@class='modal']//button[text()='Close']	# Close button in modal
//div[@role='dialog']//input[@type='text']	# Input in dialog
//*[@role='alertdialog']	# Alert dialog

Data Tables

//table//tr[2]//td[1]	# First column, second row
//table//tr[td[contains(., 'John')]]	# Row containing 'John'
//table//tr[contains(@class, 'selected')]	# Selected rows
//table//th[contains(text(), 'Name')]/following-sibling::td	# Data in column

Navigation

//nav//a[contains(@href, '/profile')]	# Profile link in nav
//menu//li[1]//a	# First menu item link
//ul[@class='breadcrumb']//li	# Breadcrumb items

Forms with Labels

//label[contains(text(), 'Email')]/following-sibling::input	# Input by label
//label[contains(text(), 'Password')]/following::input[1]	# Password after label

19. PLAYWRIGHT SPECIFIC USAGE

Playwright XPath Locator Syntax

```
// Basic XPath
page.locator("//button[@id='submit']")

// With text
page.locator("//button:has-text('Login')")

// Combined
page.locator("//form[@id='myForm']//input[@type='email']")

// Complex
page.locator("//button[contains(@class, 'primary')][1]")
```

Example in Test Code

```
import { test } from '@playwright/test';

test('XPath selectors example', async ({ page }) => {
  // By ID
  await page.locator("//input[@id='firstName']").fill("John");

  // By attribute
  await page.locator("//input[@name='email']").fill("john@test.com");

  // By text
  await page.locator("//button[text()='Submit']").click();

  // Complex
  await page.locator("//form[@id='loginForm']//button[@type='submit']").click();
});
```

20. COMMON XPATH MISTAKES TO AVOID

✗ Wrong	☑ Correct	Reason
//button[class='btn']	//button[@class='btn']	Must use @ for attributes
//button['click']	//button[text()='click']	Need text() function
//div/button	//div//button	Use // for any descendant
//button[1] (in list)	(//button)[1]	Use parentheses for first in all
contains(text, 'hello')	contains(text(), 'hello')	text must be a node
//button[@id == 'submit']	//button[@id='submit']	Use = not ==
//button and @class	//button[@class]	Predicate in brackets

21. XPATH FUNCTIONS COMPATIBILITY

Function	XPath 1.0	XPath 2.0	Selenium	Playwright	Notes
text()	☑	☑	☑	☑	Get element text
contains()	☑	☑	☑	☑	Universal support
starts-with()	☑	☑	☑	☑	Universal support
ends-with()	✗	☑	⚠	⚠	Limited support
substring()	☑	☑	☑	☑	Universal support
concat()	☑	☑	☑	☑	Universal support
normalize-space()	☑	☑	☑	☑	Universal support
count()	☑	☑	☑	☑	Universal support
position()	☑	☑	☑	☑	Universal support
last()	☑	☑	☑	☑	Universal support
lower-case()	✗	☑	⚠	⚠	Browser dependent
upper-case()	✗	☑	⚠	⚠	Browser dependent

22. TIPS & BEST PRACTICES

1. Prefer IDs when available — Faster and more reliable

```
//input[@id='email'] // Good
//form//div//input // Avoid
```

2. Use text() for buttons — More stable than index-based

```
//button[text()='Submit'] // Good
//button[1] // Avoid
```

3. Avoid deep nesting — Less maintainable

```
//form//input[@type='email'] // Good
//div//div//div//input // Avoid
```

4. Use attributes for specificity — More unique

```
//input[@data-testid='email-input'] // Best
//input[@name='email'] // Good
//input[1] // Avoid
```

5. Combine conditions wisely — More readable

```
//input[@type='text' and @required] // Good
//input[@type='text'][1] // Depends on context
```

23. QUICK REFERENCE TABLE

Purpose	XPath	Alternative
By ID	//input[@id='email']	-
By class	//div[@class='container']	//div[contains(@class, 'container')]
By name	//input[@name='email']	-
By type	//input[@type='text']	-
By text	//button[text()='Click']	//button[contains(text(), 'Click')]
By multiple attributes	//input[@type='text' and @name='email'] -	
By position	//button[1]	(//button)[1]
By contains attribute	//div[contains(@id, 'btn')]	-
Parent element	//button/..	//button/parent::div
Next sibling	//input/following-sibling::button	-
Previous sibling	//button/preceding-sibling::input	-
By child count	//form[count(input)=5]	-

Last Updated: 2026-07-16
Valid for: Playwright, Selenium, Appium
XPath Standard: XPath 1.0 (with XPath 2.0 notes)